

C++ 프로그래밍

클래스 (Class) 입문

구조체에서 클래스로

C의 구조체는 데이터만 묶었다. C++의 클래스는 데이터와 함수를 함께 묶는다.

```
1 // C 스타일 - 데이터만
2 struct Point {
3     int x;
4     int y;
5 };
6 void print_point(Point p) { printf("(%d, %d)\n", p.x, p.y); }
7
8 // C++ 스타일 - 데이터 + 함수를 하나의 타입으로
9 class Point {
10 public:
11     int x;
12     int y;
13     void print() { printf("(%d, %d)\n", x, y); }
14 };
```

클래스는 ****관련 데이터(멤버 변수)****와 ****관련 동작(멤버 함수)****을 하나의 타입으로 묶는 C++의 핵심 추상화 도구다.

클래스 선언 기본 구조

선언

```
class Student {  
public:  
    // 멤버 변수 (member variable)  
    int age;  
    char name[32];  
  
    // 멤버 함수 (member function)  
    void greet() {  
        printf("안녕하세요, %s입니다.\n", name);  
    }  
};
```

- `class` 키워드 + 클래스 이름
- `public:` — 외부에서 접근 가능
- 세미콜론으로 끝남

사용

```
int main() {  
    Student s;           // 객체(object) 생성  
    s.age = 20;  
    strcpy(s.name, "홍길동");  
  
    s.greet();           // 멤버 함수 호출  
    // 출력: 안녕하세요, 홍길동입니다.  
  
    Student* p = &s;  
    p->greet();          // 포인터로 호출  
}
```

클래스 타입의 변수를 **객체(object)** 라고 부른다.

접근 지정자 (Access Specifier)

멤버를 공개할지 숨길지 결정한다.

public / private

```
class BankAccount {  
public:  
    // 외부에서 접근 가능  
    void deposit(int amount) {  
        if (amount > 0)  
            balance += amount;  
    }  
    int get_balance() { return balance; }  
  
private:  
    // 외부에서 직접 접근 불가  
    int balance = 0;  
};
```

사용 예

```
BankAccount acc;  
  
acc.deposit(1000);           // ✅ public  
printf("%d\n", acc.get_balance()); // ✅ public  
  
// acc.balance = 9999;      // ❌ 컴파일 오류!  
//   → private 멤버에 직접 접근 불가
```

지정자

접근 범위

public

어디서든 접근 가능

private

클래스 내부에서만 접근 가능

protected

클래스 내부 + 파생 클래스

생성자와 소멸자

객체가 생성될 때 자동으로 호출되는 함수가 생성자, 소멸될 때 호출되는 함수가 소멸자다.

생성자 (Constructor)

```
class Student {
public:
    int age;
    char name[32];

    // 생성자 - 클래스 이름과 동일, 반환형 없음
    Student(int a, const char* n) {
        age = a;
        strcpy(name, n);
        printf("%s 생성!\n", name);
    }

    // 기본 생성자 (인자 없음)
    Student() : age(0) {
        strcpy(name, "이름없음");
    }
};
```

소멸자 (Destructor)

```
class Student {
public:
    // ...

    // 소멸자 - ~클래스이름, 인자 없음
    ~Student() {
        printf("%s 소멸!\n", name);
    }
};

int main() {
    Student s1(20, "홍길동"); // 생성자 호출
    Student s2;               // 기본 생성자 호출
    // main 종료 시 소멸자 자동 호출
}
```

멤버 초기화 리스트

생성자에서 멤버 변수를 초기화하는 **권장** 방법이다.

일반 초기화 (대입)

```
class Point {
public:
    int x;
    int y;

    Point(int a, int b) {
        x = a; // 생성 후 대입
        y = b;
    }
};
```

- 기본 생성 → 대입의 2단계 발생

초기화 리스트 (권장)

```
class Point {
public:
    int x;
    int y;

    // : 멤버(값), 멤버(값) 형식
    Point(int a, int b) : x(a), y(b) {}
};

int main() {
    Point p(3, 4);
    printf("(%d, %d)\n", p.x, p.y); // (3, 4)
}
```

- 생성과 동시에 초기화 (1단계)
- `const` 멤버, 참조 멤버는 반드시 초기화 리스트 사용

this 포인터

멤버 함수 안에서 자기 자신 객체를 가리키는 포인터다.

매개변수 이름 충돌 해결

```
class Student {
public:
    int age;
    char name[32];

    // 매개변수와 멤버 이름이 같을 때
    void set(int age, const char* name) {
        this->age = age;           // 멤버 변수
        strcpy(this->name, name); // 멤버 변수
    }
};
```

- `this->age` — 객체의 멤버 변수
- `age` — 매개변수 (지역 변수)

메서드 체이닝 (Method Chaining)

```
class Builder {
public:
    int x = 0;
    int y = 0;

    Builder& set_x(int v) { x = v; return *this; }
    Builder& set_y(int v) { y = v; return *this; }
};

int main() {
    Builder b;
    b.set_x(10).set_y(20); // 체이닝
    printf("(%d, %d)\n", b.x, b.y); // (10, 20)
}
```

캡슐화 (Encapsulation)

데이터를 숨기고, 함수를 통해서만 접근하게 만드는 객체지향의 핵심 원칙이다.

```
class Temperature {
public:
    // 공개 인터페이스
    void set_celsius(double c) { celsius = c; }
    double get_celsius() const { return celsius; }
    double get_fahrenheit() const { return celsius * 9.0 / 5.0 + 32.0; }

private:
    double celsius = 0.0; // 구현 세부사항은 숨김
};

int main() {
    Temperature t;
    t.set_celsius(100.0);
    printf("섭씨: %.1f\n", t.get_celsius()); // 100.0
    printf("화씨: %.1f\n", t.get_fahrenheit()); // 212.0
    // t.celsius = -999; // ❌ 직접 접근 불가 → 무결성 보장
}
```

private 으로 데이터를 숨기면 내부 구현을 바꿔도 외부 코드에 영향이 없다.

요약

개념	키워드/문법	핵심
클래스 선언	<code>class 이름 { };</code>	데이터 + 함수를 하나의 타입으로
객체 생성	<code>Student s;</code>	클래스 타입의 변수
접근 지정자	<code>public / private</code>	외부 공개 여부 결정
생성자	클래스와 동일한 이름, 반환형 없음	객체 생성 시 자동 호출
소멸자	<code>~클래스이름()</code>	객체 소멸 시 자동 호출
초기화 리스트	<code>: 멤버(값)</code>	생성자에서 권장되는 초기화 방법
this 포인터	<code>this->멤버</code>	현재 객체를 가리키는 포인터
캡슐화	<code>private</code> + getter/setter	데이터 보호 및 인터페이스 분리

클래스는 C 구조체의 확장이다. 멤버 함수, 접근 제어, 생성자/소멸자가 추가되어 데이터와 동작을 하나의 단위로 안전하게 관리할 수 있다.